

Flight Control System Architecture

Multithreaded 6-DOF Aircraft Simulation

Nuri Ziya Kirtepe
Gebze Technical University

May 28, 2026

Abstract

This document outlines the software architecture of a 6-Degrees-of-Freedom (6-DOF) aircraft flight simulator. Built from scratch in C++, the project features a custom physics engine and implements Real-Time Operating System (RTOS) concepts, specifically multithreading and resource synchronization, to emulate the behavior of a modern Flight Control Computer (FCC).

1 Introduction

Transitioning from a 1-DOF sounding rocket model, this project introduces a fully multithreaded 6-DOF flight simulator. The architecture is designed to handle complex aerodynamic forces, full rotational dynamics using quaternions, and concurrent task scheduling critical for embedded systems in aerospace engineering.

2 Flight State Machine

The aircraft's operational modes are managed by a robust State Machine. This prevents invalid system operations, such as attempting a takeoff sequence before initialization is complete.

- **INIT_BOOT:** System startup, memory allocation, and initial sensor/variable calibration.
- **IDLE / STANDBY:** Systems are online, telemetries are active, but the engine throttle is disabled.
- **TAKEOFF:** Maximum thrust applied. The system monitors velocity to execute pitch-up commands at the defined rotation speed (V_R).
- **CRUISE:** Airborne state where Newton's laws of motion are actively balancing thrust against drag, and lift against weight.
- **FAIL_SAFE:** An emergency mode triggered by thread failures or anomalous data, ensuring the system defaults to a stable, predictable state.

3 RTOS and Task Scheduling Architecture

To prevent blocking operations (such as File I/O) from disrupting the physics calculations, the main execution is decoupled into three distinct threads using `std::thread`. This mimics Rate Monotonic Scheduling (RMS) principles.

3.1 Physics & Dynamics Thread (High Priority, $\sim 1000\text{Hz}$)

This thread is strictly responsible for numerical integration (Euler method) and aerodynamic calculations. It calculates lift, drag, thrust, and updates the aircraft's 'Vector3' position and 'Quaternion' orientation. It contains no blocking delays or I/O operations.

3.2 Flight Control & State Thread (Medium Priority, $\sim 100\text{Hz}$)

Responsible for reading control inputs, updating the Flight State, and calculating the desired control surface deflections (ailerons, elevators) and engine thrust.

3.3 Telemetry & Logging Thread (Low Priority, $\sim 10\text{Hz}$)

Handles all File I/O operations. It safely extracts the current state of the aircraft and logs it to a '.csv' file for post-flight analysis.

4 Data Synchronization

Given the concurrent nature of the architecture, shared resources (e.g., Position, Velocity, Orientation) are subject to race conditions.

- **Mutexes** (`std::mutex`): Applied during the read/write operations of the physical state. When the Physics Thread updates the position, it locks the mutex. The Telemetry Thread must acquire this lock before reading, ensuring data integrity.
- **Atomics** (`std::atomic`): Used for single-value state flags (like `FlightState` or system `isRunning` booleans) to allow lock-free, thread-safe state checking across all loops.

5 Conclusion

By utilizing custom mathematics without relying on external libraries and implementing a threaded execution model, this simulation serves as a highly accurate proxy for embedded avionics software development.